Cosmos EVM 트랜잭션 처리 과정

1. 트랜잭션 라이프사이클 개요

1.1 전체 플로우

```
사용자 요청 → Mempool → Ante Handler → EVM 실행 → 상태 커밋 → 이벤트 발생
```

1.2 주요 단계

1. **트랜잭션 생성**: 사용자가 MetaMask 등을 통해 트랜잭션 생성
2. **Mempool 진입**: 트랜잭션이 메모리풀에 추가
3. **블록 제안**: 밸리데이터가 트랜잭션을 블록에 포함
4. **Ante Handler**: 서명 검증 및 수수료 처리
5. **EVM 실행**: 스마트 컨트랙트 실행
6. **상태 업데이트**: 블록체인 상태 반영
7. **이벤트 발생**: 로그 및 이벤트 생성

2. Mempool 관리

2.1 EVM Mempool 구조

```
type ExperimentalEVMMempool struct {

// 논스 기반 트랜잭션 정렬
```

```
senderNonceMap  map[string]*list.List

// 가스 가격 기반 우선순위

priorityQueue *PriorityQueue

// 설정

config *EVMMempoolConfig

}
```

2.2 트랜잭션 우선순위

- **가스 가격**: 높은 가스 가격의 트랜잭션 우선 처리
- **논스 순서**: 같은 발신자의 트랜잭션은 논스 순서대로 처리
- **계정 잔액**: 충분한 잔액을 가진 트랜잭션 우선

2.3 Mempool 검증

```
func (emp *ExperimentalEVMMempool) CheckTx(ctx context.Context, req *abci.Reques

// 트랜잭션 디코딩

tx, err := emp.txDecoder(req.Tx)

if  err != nil {

return &abci.ResponseCheckTx{Code: 1}, err

}

// Ante Handler 실행

_, err = emp.anteHandler(ctx, tx, false)

if  err != nil {
```

```
return &abci.ResponseCheckTx{Code: 1}, err
}
return &abci.ResponseCheckTx{Code: 0}, nil
}
```

3. Ante Handler 처리

3.1 Ante Handler 체인

```
func  NewAnteHandler(options HandlerOptions) sdk.AnteHandler {
return  sdk.ChainAnteDecorators(
// 기본 검증
ante.NewSetUpContextDecorator(),
ante.NewExtensionOptionsDecorator(options.ExtensionOptionChecker),
ante.NewValidateBasicDecorator(),
ante.NewTxTimeoutHeightDecorator(),
ante.NewValidateMemoDecorator(options.AccountKeeper),
// 수수료 처리
ante.NewConsumeGasForTxSizeDecorator(options.AccountKeeper),
ante.NewDeductFeeDecorator(options.AccountKeeper, options.BankKeeper, options.Fe
// 서명 검증
ante.NewSetPubKeyDecorator(options.AccountKeeper),
ante.NewValidateSigCountDecorator(options.AccountKeeper),
```

```
ante.NewSigGasConsumeDecorator(options.AccountKeeper, options.SigGasConsumer),
ante.NewSigVerificationDecorator(options.AccountKeeper, options.SignModeHandler)
// EVM 특화 처리
evmante.NewEthSetUpContextDecorator(options.EvmKeeper),
evmante.NewEthMempoolFeeDecorator(options.EvmKeeper),
evmante.NewEthValidateBasicDecorator(options.EvmKeeper),
evmante.NewEthSigVerificationDecorator(options.EvmKeeper),
evmante.NewEthAccountVerificationDecorator(options.AccountKeeper, options.EvmKee
evmante.NewEthGasConsumeDecorator(options.EvmKeeper, options.MaxTxGasWanted),
evmante.NewCanTransferDecorator(options.EvmKeeper),
evmante.NewEthIncrementSenderSequenceDecorator(options.AccountKeeper),
evmante.NewGasWantedDecorator(options.EvmKeeper, options.FeeMarketKeeper),
evmante.NewEthEmitEventDecorator(options.EvmKeeper),
)
}
```

3.2 주요 Decorator 기능

3.2.1 EthSetUpContextDecorator

- EVM 실행 컨텍스트 설정
- 체인 ID 및 블록 정보 설정

3.2.2 EthMempoolFeeDecorator

- 최소 가스 가격 검증

- Fee Market 파라미터 적용

3.2.3 EthSigVerificationDecorator

- 이더리움 서명 검증
- ECDSA 서명 복구 및 검증

3.2.4 EthGasConsumeDecorator

- 가스 한도 검증
- 가스 소비량 계산

4. EVM 실행 과정

4.1 메시지 처리

```
func (k Keeper) EthereumTx(goCtx context.Context, msg *types.MsgEthereumTx) (*ty

ctx := sdk.UnwrapSDKContext(goCtx)

// 트랜잭션 데이터 추출

txData, err := types.UnpackTxData(msg.Data)

if err != nil {

return nil, err

}

// EVM 실행

res, err := k.ApplyTransaction(ctx, msg)

if err != nil {

return nil, err
```

```
}
return res, nil
}
```

4.2 EVM 상태 머신 실행

```
func (k Keeper) ApplyTransaction(ctx sdk.Context, msgEthTx *types.MsgEthereumTx)
// EVM 인스턴스 생성
evm := k.NewEVM(ctx, msgEthTx, &types.EvmTxArgs{})
// 트랜잭션 실행
leftoverGas := msgEthTx.GetGas()
if msgEthTx.To() == nil {
// 컨트랙트 배포
res, err := k.CreateContract(ctx, evm, msgEthTx)
return res, err
} else {
// 컨트랙트 호출
res, err := k.CallContract(ctx, evm, msgEthTx)
return res, err
}
}
```

4.3 컨트랙트 배포

```go
func (k Keeper) CreateContract(ctx sdk.Context, evm *vm.EVM, msg *types.MsgEther

// 컨트랙트 주소 생성

contractAddr := crypto.CreateAddress(msg.From(), msg.GetNonce())

// EVM CREATE 실행

ret, contractAddr, leftoverGas, err := evm.Create(

vm.AccountRef(msg.From()),

msg.GetData(),

msg.GetGas(),

msg.GetValue(),

)

if  err != nil {

return  nil, err

}

return &types.MsgEthereumTxResponse{

Hash: msg.Hash(),

Logs: types.NewLogsFromEth(evm.StateDB.GetLogs(msg.Hash())),

Ret: ret,

VmError: "",

GasUsed: msg.GetGas() - leftoverGas,

}, nil

}
```

4.4 컨트랙트 호출

```go
func (k Keeper) CallContract(ctx sdk.Context, evm *vm.EVM, msg *types.MsgEthereu

// EVM CALL 실행

ret, leftoverGas, err := evm.Call(

vm.AccountRef(msg.From()),

*msg.To(),

msg.GetData(),

msg.GetGas(),

msg.GetValue(),

)

if  err != nil {

return  nil, err

}

return &types.MsgEthereumTxResponse{

Hash: msg.Hash(),

Logs: types.NewLogsFromEth(evm.StateDB.GetLogs(msg.Hash())),

Ret: ret,

VmError: "",

GasUsed: msg.GetGas() - leftoverGas,

}, nil

}
```

5. 상태 관리

5.1 StateDB 인터페이스

```
type StateDB interface {
// 계정 관리
CreateAccount(common.Address)
SubBalance(common.Address, *big.Int)
AddBalance(common.Address, *big.Int)
GetBalance(common.Address) *big.Int
// 스토리지 관리
SetState(common.Address, common.Hash, common.Hash)
GetState(common.Address, common.Hash) common.Hash
// 코드 관리
SetCode(common.Address, []byte)
GetCode(common.Address) []byte
// 로그 관리
AddLog(*types.Log)
GetLogs(common.Hash) []*types.Log
}
```

5.2 Cosmos 상태와의 동기화

```
func (k Keeper) CommitStateDB(ctx sdk.Context, stateDB *statedb.StateDB) error {
```

```
// 계정 상태 커밋
for addr, account := range stateDB.GetAccounts() {
k.SetAccount(ctx, addr, account)
}
// 스토리지 상태 커밋
for addr, storage := range stateDB.GetStorages() {
k.SetStorage(ctx, addr, storage)
}
// 코드 커밋
for addr, code := range stateDB.GetCodes() {
k.SetCode(ctx, addr, code)
}
return nil
}
```

6. 이벤트 및 로그 처리

6.1 EVM 로그 생성

```
// Solidity에서 이벤트 발생
event Transfer(address indexed from, address indexed to, uint256 value);

// EVM에서 로그 생성
```

```
log := &types.Log{
Address: contractAddr,
Topics: []common.Hash{transferEventHash, fromHash, toHash},
Data: valueBytes,
}
stateDB.AddLog(log)
```

6.2 Cosmos 이벤트 변환

```
func (k Keeper) EmitEthereumTxEvent(ctx sdk.Context, txHash common.Hash, logs []
// EVM 로그를 Cosmos 이벤트로 변환
for _, log := range logs {
ctx.EventManager().EmitEvent(
sdk.NewEvent(
types.EventTypeEthereumTx,
sdk.NewAttribute(types.AttributeKeyContractAddress, log.Address.Hex()),
sdk.NewAttribute(types.AttributeKeyTxHash, txHash.Hex()),
),
)
}
}
```

7. 에러 처리 및 리버트

7.1 EVM 에러 처리

```
func (k Keeper) HandleVMError(ctx sdk.Context, err  error, gasUsed  uint64) *typ

if  err == vm.ErrExecutionReverted {

return &types.MsgEthereumTxResponse{

VmError: err.Error(),

GasUsed: gasUsed,

}

}

// 기타 에러는 전체 트랜잭션 실패

return  nil

}
```

7.2 상태 롤백

- **성공 시**: 모든 상태 변경 커밋
- **실패 시**: 가스 소비만 반영하고 나머지 상태 롤백
- **리버트 시**: 명시적 리버트 메시지와 함께 상태 롤백

이 문서는 Cosmos EVM에서의 트랜잭션 처리 과정을 상세히 설명합니다. 다음 문서에서는 IBC 통합과 크로스체인 기능을 살펴보겠습니다.